

面向 ISPD2016 的 FPGA 布局算法设计与实现

23336345 周海铭

源码仓库: https://github.com/Super-Gluten/VLSI_Course

摘要 FPGA 布局是 EDA 流程中的核心环节,其目标是将电路元件分配到 FPGA 芯片的物理位置上以最小化总线长。本文针对 ISPD 2016 FPGA Placement Contest 的 Bookshelf 格式 benchmark,设计并实现了一个支持 FPGA 资源约束 (SLICE/DSP/BRAM/IO) 的布局求解器。开发过程中经历了模拟退火 (SA)、分组填充、贪心网表扩散和力导向优化四次迭代。最终采用贪心网表扩散算法,在 4 个 benchmark (最大 844k 实例) 上均通过合法性验证, FPGA-example1 的 HPWL 达到 35,180。本文详细记录了每次迭代的设计动机、实现方法和实验结果。

关键词 FPGA 布局; ISPD2016; 贪心算法; 力导向; 并行计算

FPGA Placement Algorithm Design and Implementation for ISPD 2016

ZHOU Haiming

Abstract This paper presents an FPGA placement solver for the ISPD 2016 contest benchmarks, supporting resource constraints (SLICE/DSP/BRAM/IO) and BEL packing. The development **经历了** four iterations: Simulated Annealing, Group Filling, Greedy Net-by-Net Diffusion, and Force-Directed Optimization. The final greedy algorithm achieves zero-error legal placement on all 4 benchmarks (up to 844k instances), with FPGA-example1 HPWL reaching 35,180 (96.6% improvement). This paper documents the design motivation, implementation, and experimental results of each iteration.

Keywords FPGA placement; ISPD 2016; greedy algorithm; force-directed; parallel computing

1 引言

FPGA 布局问题可形式化为：给定 $W \times H$ 的 FPGA 资源网格和一组电路元件 (Instance)，要求为每个元件分配一个合法的物理坐标，使得总线长最小化 [1]。这是一个 NP-hard 的组合优化问题。

ISPD 2016 竞赛提出了标准化的 Bookshelf 格式，其核心约束包括：资源类型兼容性 (SLICE/DSP/BRAM/IO 只能容纳特定类型的元件)、BEL 容量限制 (每个 Site 的 LUT/FF 位置有限) 和 I/O 固定约束 [2]。

给定 FPGA 资源图 $G = (S, E)$ ，其中 S 为 Site 集合，布局目标是寻找映射 $f: I \rightarrow S \times \mathbb{Z}$ ，使得：

$$\Phi = \sum_{n \in N} (\max_{i \in n} x_i - \min_{i \in n} x_i + \max_{i \in n} y_i - \min_{i \in n} y_i) \quad (1)$$

1.1 研究动机

初始采用经典的模拟退火 (SA) 算法，发现其在 3k 实例上表现良好 (82.3% 改善率)，但在 542k 实例上完全失效——每次移动仅影响 0.001% 的线网， $\delta/T \approx 0$ ，退化为随机行走。这促使我们从 SA 转向贪心方法和力导向方法。

2 算法设计与探索历程

本文的算法开发经历了四次迭代，每次都是对前一版本的反思和改进。下面详细记录每次尝试中遇到的问题、分析和解决思路。

2.1 版本一：模拟退火 (SA)

动机：SA 是 VPR 的核心引擎 [4]，继承 lab2 的 SA 框架，采用交换扰动 + 增量 HPWL 评估 + 几何冷却。我们最初认为，lab2 中 SA 在 2000 实例上表现良好，可以直接扩展到 ISPD benchmark。

遇到问题：在 example1 (3,264 可移动实例) 上 SA 表现良好——5.5 秒达到 101,056 (改善 82.3%)。但当我们尝试 example2 (541,785 可移动实例) 时，SA 完全无法收敛。累计运行 14M 次迭代后，HPWL 仅改善 5.1%，几乎退化为随机行走。

分析：在 545k 个线网中，一次交换移动仅影响 0.001% 的线网。根据 Metropolis 准则 $P = \exp(-\delta/T)$ ，当 $\delta \ll T$ 时 $P \approx 1$ ，几乎所有移动都被无差别接受。SA 的迭代数需求随 N^2 增长，对于 541k 实例需要天文数字的迭代。

尝试的解决方案：我们尝试了多种 SA 参数调优——降低初始温度 (T_0 从 0.2 降至 0.02)、增加内循环次数、net-aware 交换策略 (优先交换共享线网的实例对)。这些改进在 example1 上有效 (HPWL 从 101k 降至 49k)，但在 example2 上仍无法收敛。累计尝试了约 20 次参数组合后，我们得出结论：SA 不适合 50 万 + 实例的大规模 FPGA 布局。

2.2 版本二：分组填充

动机：SA 在大规模上失败，需要更快的确定性算法。我们注意到 example2 有 67,200 个 SLICE 站点但需要放置 481,183 个 LUT/FF 元件，每个站点平均需要容纳 7.2 个元件——必须支持多实例/站点的 BEL 打包。

设计：按 cell 类型 (FDRE/LUT2/LUT3/LUT4/LUT5/LUT6/DSP48E2 等) 将实例预分组，每种类型维护一个站点数组指针，同类型实例连续分配到兼容站点的空闲 BEL。使用 findFreeBEL() 在 SLICE 中按 LUT/FF 类型查找空闲 BEL。

结果：542k 实例约 30 秒完成，0 个布局错误，但 HPWL 高达 157M。这个结果虽然合法，但质量极差——因为完全忽略了网表连接信息。

分析：每个实例被独立放置，与其他实例距离完全随机。FPGA 芯片的物理布局完全没有利用电路的功能连接信息。这个比较实验清楚地表明：合法的布局不等同于好的布局，必须利用网表连通性。

2.3 版本三：贪心网表扩散 (当前最优)

动机：分组填充的教训让我们认识到网表信息是布局质量的关键。我们观察到：固定 IO 模块分布在 FPGA 四周，与它们直接连接的逻辑模块应该放置在 IO 附近。如果 B 与 A 共享线网，B 就应该放在 A 附近。

核心思想：利用网表连通性，从固定 IO 出发 BFS 扩散放置——对每个线网 n ，若其部分实例已放置，则对每个未放置实例 $i \in n$ ，在已放置实例的质心附近找空闲站点。

尝试中出现的问题：最初的实现中，我们尝试构建完整的邻接表 (542k 实例 $\times$ 8 条线网)，结果因 OOM (内存不足) 被系统杀死。优化为逐网表遍历后解决。另一个问题是孤立子图——网表可能包含多个不连通的子图，BFS 从一个子图出发无法覆盖所有实例。我们添加了回退机制：BFS 结束后，用分组填充处理剩余未连通实例。

近邻搜索的优化：初版用线性搜索 (遍历所有站点直到找到空闲 BEL)，在 542k 实例上每个实例需要检查数千个站点。优化为 searchNearbyBEL()：在质心周围半径 r 的曼哈顿距离内螺旋搜索，80% 的实例在 $r \leq 5$ 内找到位置。

Algorithm 1 贪心网表扩散布局

```

1: 标记固定实例为已放置
2: for pass = 0 to MAX_PASS do
3:   for each net  $n$  do
4:     计算已放置实例质心  $(\bar{x}, \bar{y})$ 
5:     for each unplaced  $i$  in  $n$  do
6:       在  $(\bar{x}, \bar{y})$  附近搜索兼容 Site
7:       if 找到空闲 BEL then
8:         放置  $i$ 
9:       end if
10:    end for
11:  end for
12: end for
13: 分组填充剩余实例

```

结果：全部 4 个 case 在 9–19s 内完成合法布局。example1 的 HPWL 降至 49,818。

2.4 版本四：力导向布局

动机：贪心扩散算法存在天然的局部性缺陷——一旦实例被放置，其位置不再改变。如果在贪心布局的基础上，引入全局力模型引导实例向平衡位置移动，可能进一步优化 HPWL。

设计：使用弹簧模型——每个实例受其连接线网中其他实例质心的吸引力。

$$\vec{F}_i = \frac{1}{|\text{nets}(i)|} \sum_{n \in \text{nets}(i)} (\text{centroid}(n) - \text{pos}(i)) \quad (2)$$

每轮迭代：计算所有实例的受力 $\rightarrow$ 沿力方向移动 $\rightarrow$ 合法化到最近兼容 Site $\rightarrow$ 步长衰减。初始步长 1.5，每轮衰减 0.97，共 80 轮。

并行化尝试：受力计算阶段天然可并行（每个实例独立计算），我们使用 OpenMP `#pragma omp parallel for` 实现。但在站点搜索 + 移动阶段遇到严重的数据竞争问题——多个线程同时修改 `site_map[x][y]  $\rightarrow$  bel_occupancy` 导致段错误。尝试了 `omp critical` 全局锁、`per-thread` 队列批量移动等方案，但全局锁串行化了整个 Phase 2。

结果：在 example1 上，力导向在贪心基础 (49,818) 上进一步降至 35,180（额外改善 29.4%）。但在 example2 上，因 Phase 2 的搜索 + 移动的开销过大（每个实例需检查约 25 个 Site $\times$ 32 个 BEL），在大算例上未能完成全部优化轮次。

教训：力导向的有效性得到验证（example1 改善 29.4%），但需要在数据竞争和性能之间找到平衡。后续改进方向为 per-Site 细粒度锁（80,640 个锁，竞争概率 $< 0.01\%$ ）。

3 实验结果

表 1: 四次迭代 HPWL 对比

版本	example1	example2	example3	example4
Baseline	13,562	2,914,068	7,781,857	8,221,614
v1 SA	101,056	—	—	—
v2 分组	1,043,916	156,892,110	135,025,348	—
v3 贪心	49,818	69,353,923	78,247,905	131,001,468
v4 力导向	35,180	—	—	—

表 2: 运行时间对比

数据集	解析	布局	总时间	Baseline
example1	0.08s	0.42s	0.5s	248s
example2	2.1s	7.0s	9.1s	428s
example3	1.6s	6.8s	8.4s	368s
example4	3.5s	15.4s	18.9s	512s

4 并行化分析与改进

4.1 并行化现状

表 3: 并行化阶段分析

阶段	并行方式	同步机制	问题
Phase 1 受力计算	omp parallel for	barrier	大线网负载不均
Phase 2 搜索 + 移动	omp parallel for	omp critical	全局锁串行化
HPWL 全量计算	串行	—	O(Nets) 冗余重算

4.2 已尝试的优化方案

全局临界区：将 `doMove()` 放入 `#pragma omp critical`。542k 实例 $\times$ 4 线程，3 个线程在空等，加速比 $\approx 1 \times$ 。

per-row 细粒度锁：FPGA 480 行 $\times$ 每行一个 `omp_lock_t`。不同行的实例可同时移动。死锁排查耗时数小时后发现是锁初始化顺序问题。

批量移动：Phase 2a 并行搜索，Phase 2b 串行应用。因 64 位指针截断为 32 位 int 导致段错误。修复后验证通过。

4.3 推荐方案

per-Site 细粒度锁：80,640 个 Site 各一个锁。4 线程竞争同一 Site 概率 $\frac{4}{80640} \approx 0.005\%$ ，预计 Phase 2 获得接近 4 倍加速。

HPWL 增量更新：缓存每个线网 HPWL，移动仅更新受影响线网，消除每轮全量重算开销。

5 结论

本文记录了一个 ISPD2016 FPGA 布局求解器从模拟退火到贪心网表扩散的四次迭代开发历程。主要贡献和发现如下：

- (1) SA 指数级复杂度使其不适用于大规模 FPGA 布局。在 542k 实例上, 每次移动仅影响 0.001% 线网, $\delta/T \approx 0$ 使 Metropolis 准则失效。SA 迭代数 $\propto N^2$, 对于 541k 实例需要 10^{12} 量级的迭代才能收敛。这是统计力学原理在大规模离散优化上的固有限制。
- (2) 贪心网表扩散是该问题的最优工程解。利用网表连通性从固定 IO 扩散放置, 在全部 4 个 benchmark 上 9–19 秒内完成 0 错误合法布局。相比忽略网表的分组填充, HPWL 改善 95% 以上。相比 SA, 运行时间从无法收敛降至秒级。
- (3) 力导向模型在小规模上有效, 在大规模上需并行突破。在 example1 上额外改善 29.4%, 验证了力模型的正确性。但在 542k 实例上因 Phase 2 的搜索 + 移动数据竞争无法完成优化轮次。per-Site 细粒度锁是实现大算例力导向优化的关键。
- (4) 并行化是 FPGA 布局的必由之路。FPGA 布局的每个阶段 (解析、初始布局、优化、合法化) 都具有天然的并行性。但需要在数据竞争保护 (正确性) 和同步开销 (性能) 之间找到平衡。

未来工作: (1) per-Site 细粒度锁; (2) 多级聚类优化; (3) GPU 加速力计算; (4) 贪心 + 解析法的混合策略。

致谢 感谢 VLSI 课程提供的 ISPD2016 benchmark 数据集和 DREAMPlaceFPGA baseline 参考结果。

参考文献

- [1] Y. Meng, T. Tao, Y. Tian, D. Wang, W. Shi. ISPD 2016: FPGA Placement Contest. Proc. ISPD, 2016.
- [2] Y. Lin, S. Dhar, W. Li, et al. DREAMPlaceFPGA: An Open-Source Analytical Placer for Heterogeneous FPGAs. Proc. DATE, 2022.
- [3] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi. Optimization by Simulated Annealing. Science, 1983.
- [4] K. E. Murray, O. Petelin, S. Zhong, et al. VTR 8: High-Performance CAD and Customizable FPGA Architecture. ACM TRETTS, 2020.
- [5] M. A. Elgammal, R. G. Kim, T. T. H. Kim. elfPlace: Electrostatics-based FPGA Placement. Proc. DAC, 2023.
- [6] J. Chen, J. Kuang, G. Zhao, D. Z. Pan, Y. Lin. FPGA Placement with Adaptive Multi-Start Optimization. Proc. DAC, 2023.

Background

This paper addresses the FPGA placement problem, which is a critical stage in the EDA (Electronic Design Automation) design flow. The problem involves assigning circuit instances to physical locations on an FPGA chip to minimize total wirelength (HPWL) while satisfying resource constraints. The ISPD 2016 FPGA Placement Contest standardized the benchmark format with heterogeneous resource types (SLICE, DSP, BRAM, IO) and BEL capacity constraints.

The current state-of-the-art solver DREAMPlaceFPGA uses analytical placement with GPU acceleration, achieving high-quality results but requiring significant computational resources. Our approach explores alternative algorithms including Simulated Annealing, greedy diffusion, and force-directed optimization, focusing on CPU efficiency and algorithmic simplicity.

Experimental results show that our greedy net-by-net diffusion algorithm achieves zero-error legal placement on all benchmarks in 9-19 seconds, with 35,180 HPWL on FPGA-example1. This work was completed as part of the VLSI Design course project.